

agent-reliability: project FAQ

A tool-using LLM agent plus a framework for studying where it fails on multi-step tasks. Prepared as an interview and explanation aid. Every number here comes from real benchmark runs (Gemma 4 31B via the Gemini API, 12-13 September 2026); none are estimated.

github.com/aadycm/agent-reliability

The 30-second summary

I built an AI agent that solves multi-step problems by calling tools: a calculator, a sandboxed Python executor, a file reader and a search engine. Around it I built a framework to measure where such agents fail: 23 benchmark tasks with verified answers, full step-by-step traces, automatic failure classification, and on/off ablations of two common reliability techniques - reflection (plan before acting) and self-check (verify before answering). The study ran 92 real agent runs.

Headline results

Condition	Correct	Model calls/task	Tokens/task	vs baseline	Tool errors
baseline	23/23	4.2	6,331	1.0x	0
self-check	23/23	5.3	9,017	1.4x	0
reflection	23/23	6.0	14,090	2.2x	6 (in 5 tasks)
reflection + self-check	23/23	7.0	18,091	2.9x	5 (in 5 tasks)

95% Wilson confidence interval for 23/23 is 85.7%-100%. 0 runs lost to API errors. Wall-clock time is dominated by rate-limit and retry waits, so cost is compared in tokens and model calls.

The five findings

1. Ceiling effect: no failures to study.

The agent solved every task in every condition, including the 5+ step tasks and the designed traps. The failure taxonomy had nothing to classify and the interventions had no room to improve accuracy. The honest conclusion is that this benchmark is too easy for this model.

2. Self-check cost tokens and changed nothing.

Across 45 verification prompts the model re-submitted the same answer every time and never called a tool to re-verify. Cost: 1.4x baseline tokens.

3. Reflection made tool use worse without changing accuracy.

In both reflection conditions the same 5 file-based tasks (T04, T08, T09, T10, T20) hit tool errors: planning before seeing any tool output, the model wrote Python that opened the CSV/log files directly, which the sandbox blocks. Without reflection it used the file-reader tool first, as intended. It recovered every time, but cost 2.2x baseline tokens. A plan written blind can pick the wrong mechanism.

4. The first self-check implementation silently failed - the most useful finding.

Delivering the verification request as a text part beside the tool results produced empty model replies in 45 of 46 runs. Those runs still scored 23/23, because the loop nudged the model into re-submitting, so accuracy alone could never have revealed it. Only the traces did. Fixed by putting the request inside the final_answer tool result; reports now show 'self-checks answered / sent' so a skipped intervention is visible.

5. Minor format slips.

2 of 92 runs answered in plain text instead of calling the `final_answer` tool. Both were graded correctly from their explicit final line. This also exposed a gap: self-check cannot trigger on plain-text answers.

What the study does NOT show

- It does not show that reflection or self-check never help - only that they did not help on tasks this model already solves.
- 23 tasks, one run per condition: a small sample with wide confidence intervals.
- One model, one prompt set. Results may not transfer to other models or prompt wordings.
- Tasks, grading rules and failure heuristics were written by the same author.
- Timing is not a fair efficiency measure because of API retries and throttling.

How it works (the technical core)

The agent loop

1. Send the task plus tool schemas to the model. 2. The model replies with reasoning text and/or a structured function call, e.g. `search(query="Veltrix Dynamics")`. 3. The framework runs that tool and returns the result. 4. Repeat until the model calls `final_answer(...)` or hits the 12-step limit. Runs can also end as `step_limit`, `no_answer` or `api_error`.

- **Automatic function calling is off.** The SDK can run tools for you, but then individual steps cannot be logged or studied; running them in-framework is what makes the traces possible.
- **Tool errors are returned to the model** as `{"error": ...}` rather than crashing the run, so the agent can recover - and the evidence stays in the trace. All 11 tool errors in the study were recovered from.
- **A dedicated `final_answer` tool** yields a clean, gradable answer instead of parsing prose.
- **Interventions are hooks in the same loop:** reflection adds a tool-less planning call before execution; self-check intercepts the first `final_answer` and asks the model to verify it. The step limit is extended by the number of self-checks, so the ablation measures verification, not a smaller step budget.

Tools

Tool	Purpose	Engineering note
calculator	arithmetic	AST walker, never <code>eval()</code>
python_exec	run a short program	separate process, 10s timeout, blocked imports; not a security boundary
read_file / list_files	read benchmark files	path traversal rejected
search	mock knowledge base	deterministic keyword scoring

New tools are registered with a decorator: a function, a description and a JSON schema.

Benchmark design

- **23 tasks:** 10 easy (2 steps), 8 medium (3-4), 5 hard (5+ steps), where steps = tool calls in the canonical solution.
- **Fictional world** (invented companies, towns, people, currency): the model cannot know these facts from pretraining, so correct answers require real tool use and unsupported answers are detectably invented. This also rules out training-data contamination.

- **Deliberate traps:** two similarly named companies; an INFO log line containing the word ERROR; a recipe that serves 3 not 4; highest unit price versus highest total revenue.
- **Verified answers:** a test re-derives every answer with the real tools and checks that each declared intermediate value actually appears in the tool outputs.
- **Grading:** numeric tolerances; answers containing more than two numbers count as hedging and fail; whole-word matching so "19" does not satisfy "9"; free-text answers are graded on their final line.

Failure taxonomy (automatic, with manual override)

Category	Primary heuristic signal
infinite_loop	step limit reached while repeating an identical call 3+ times
gave_up_early	no answer, or a refusal phrase such as "cannot determine"
hallucinated_tool_result	answered without the information tools, or used a number no tool returned
wrong_tool_choice	a required tool never called while other tools were used
bad_plan	right tools but required information never retrieved, or correct facts combined with wrong logic
execution_error	near miss ($\leq 2\%$), malformed answer, or unrecovered tool errors

api_error is tracked separately as infrastructure, never as an agent failure. Manual labels override heuristics, and the report shows heuristic-vs-manual agreement.

Reliability and statistics

- **Exponential backoff with jitter** on 429/5xx/timeouts, honouring the server's suggested retry delay, plus client-side rate limiting.
- **Daily-quota detection:** Google's per-day quota error still says "retry in 30s", so the code reads the quota id and stops the run cleanly instead of retrying all day.
- **Resumable runs:** each completed trace is written immediately, so an interrupted run continues where it stopped.
- **58 offline tests**, including a scripted fake LLM that exercises the loop, retries, self-check and failure heuristics with no API calls.
- **Wilson 95% intervals** (well behaved near 100%, unlike the normal approximation) and a **paired exact McNemar test** comparing conditions on the same tasks, counting only the tasks whose outcome flipped.

Questions you may be asked, with answers

Basics

What is an AI agent, and how does it differ from a chatbot?

A chatbot answers in a single reply. An agent runs in a loop: it chooses actions, calls tools, reads the results and continues until the task is done or a limit stops it.

What is function calling?

You give the model a JSON schema for each tool; it returns structured calls with arguments instead of free text. Your code executes the tool and feeds the result back. It is far more reliable than parsing prose for commands.

Why Gemma rather than Gemini for the study?

The Gemini Flash free tier allowed 20 requests per day and one condition needs roughly 100. Gemma 4 31B was free, supported function calling and is open-weights, which also helps reproducibility. The backend is one class, so switching models is a one-line change.

Design decisions

Why fictional data in the knowledge base?

To prevent training-data contamination and to force genuine tool use. If the model answers without searching, the answer must be invented - which makes hallucination detectable rather than a judgement call.

Why not let the SDK call the tools automatically?

Automatic function calling hides the individual steps. Running tools in the framework is what produces the traces the whole study depends on.

Why return tool errors to the model instead of failing the run?

Real agents must recover from errors, and the failed call stays in the trace as evidence. Every tool error in the study was recovered from.

How do you know your benchmark answers are correct?

A test re-derives all 23 answers using the real tools, and checks that each task's declared intermediate values appear in the tool outputs. No answer is hand-computed and unchecked.

How do you distinguish a bad plan from an execution error?

Via the declared intermediates. If the needed facts were never retrieved, it is a bad plan; if they were retrieved and the final answer is still wrong, it is an execution error.

How can you detect a hallucinated tool result automatically?

Two signals: an answer produced without ever calling the information tools, and numbers appearing in calculator or Python arguments that never appeared in any tool output or in the task text (ignoring small numbers and constants, and tolerating rounding).

Results and statistics

Everything scored 100%. Is that a failed experiment?

No. It is a valid, reportable result: the benchmark is too easy for this model. The study still produced real findings about intervention cost, reflection degrading tool use, and a silently broken intervention. The framework is ready for harder settings, which is the documented next step.

Why Wilson confidence intervals?

The standard normal approximation gives a zero-width interval at 100%, which is misleading. Wilson gives 85.7%-100% for 23/23 - honest about a small sample.

Why McNemar rather than comparing two percentages?

The same tasks are run in both conditions, so the observations are paired. McNemar uses only the discordant pairs (tasks fixed or broken by the intervention), which is the correct test for paired binary outcomes.

So is reflection harmful?

In this setting it cost 2.2x the tokens and introduced tool errors without improving accuracy. That does not generalise to harder tasks or other models, where planning may well help. The claim is scoped to what was measured.

How did you find the self-check bug?

By reading the traces: self-check steps had been relabelled as nudges and had zero output tokens. Counting them gave 45 of 46. Accuracy could not have revealed it, because it stayed at 100%.

Engineering

How did you handle rate limits and API failures?

Client-side request spacing, exponential backoff with jitter, honouring server-suggested delays, distinguishing per-minute from per-day quotas, and resumable runs. Unrecoverable failures are recorded as `api_error`, excluded from success rates and retried on resume.

Is the Python sandbox secure?

No, and the README says so plainly. It has an import/call blocklist, an isolated subprocess, a temp working directory, a 10s timeout, output truncation and POSIX resource limits. It protects against accidents, not a determined adversary; real isolation would need containers or a VM.

How do you test code that depends on an LLM?

With a scripted backend that replays fixed responses, so the loop, retries, interventions and failure heuristics are tested deterministically and offline. 58 tests run with no API key.

What if the model answers in plain text instead of calling `final_answer`?

The text is accepted but flagged as `finish_mode=text_response` and graded on its final line. It is a known gap: self-check cannot intercept that path. It happened in 2 of 92 runs.

Critical questions

Is 23 tasks not too small, and author-written?

Yes on both counts, and both are listed as limitations. That is why confidence intervals and a paired test are reported rather than bare percentages, and why harder, more varied tasks are the stated next step.

What would you do differently next time?

Pilot on a weaker model first to avoid the ceiling effect; verify that each intervention is actually delivered before running a full condition; use repeats at non-zero temperature to measure variance; and have a second annotator label failures to measure agreement.

How much of this did you build yourself?

Answer honestly, including any AI assistance, then demonstrate understanding: walk through the loop, the trap tasks, and the self-check bug. Understanding the system is what the question is really testing.

Source: `reports/gemma/summary.md` and the raw per-task traces in `runs/`, both committed to the repository.